

Purpose of the Code:

This code defines a **model training pipeline** that:

1. Trains multiple regression models.
2. Tunes their hyperparameters.
3. Evaluates their performance.
4. Saves the best-performing model.

Key Components:

1. ModelTrainerConfig

@dataclass

class ModelTrainerConfig:

trained_model_file_path = os.path.join("artifacts", "model.pkl")

- A dataclass to store the path where the trained model will be saved.

2. ModelTrainer Class

Handles the actual model training process.

initiate_model_trainer(train_array, test_array):

This method:

- **Splits** the data into features and target (X_train, y_train, etc.).
- **Initializes models** (RandomForest, XGBoost, CatBoost, etc.).
- **Defines hyperparameter grids** for each model.
- **Calls evaluate_models()** (presumably a utility function you wrote) to:
 - Train all models using GridSearch or similar.
 - Evaluate their R² scores.
- **Finds the best model** based on highest score.
- **Saves** the best model using save_object().
- **Returns** the R² score of the best model on test data.

Outputs:

- Saves the best model as artifacts/model.pkl
- Returns the R² score of the best model on the test set.

Dependencies (in src.utils):

- evaluate_models() – Trains and evaluates models with given params.
- save_object() – Serializes the best model to disk.
- CustomException – Custom error handling.
- logging – For debugging/logging steps.

line-by-line explanation

```
import os
import sys
from dataclasses import dataclass
```

Imports basic Python modules:

os: For handling file paths.

sys: For accessing system-specific parameters (used in error handling).

dataclass: Used to define simple configuration classes with less boilerplate.

```
from catboost import CatBoostRegressor
from sklearn.ensemble import (
    AdaBoostRegressor,
    GradientBoostingRegressor,
    RandomForestRegressor,
)
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from xgboost import XGBRegressor
```

Imports various machine learning models and tools:

From libraries: CatBoost, XGBoost, and scikit-learn.

Models include: Decision Tree, Random Forest, Linear Regression, Gradient Boosting, etc.

r2_score: Used to evaluate model accuracy.

```
from src.exception import CustomException
from src.logger import logging
```

Custom modules (likely created by you or your team):

CustomException: Custom error class for better error handling.

logging: Custom logging setup to log training steps.

```
from src.utils import save_object, evaluate_models
```

✔ Utility functions:

save_object: Serializes and saves Python objects (e.g., trained models) to disk.

evaluate_models: Function that trains, tunes, and evaluates different models.

📁 Configuration Class

@dataclass

class ModelTrainerConfig:

trained_model_file_path=os.path.join("artifacts","model.pkl")

✔ Defines config class using @dataclass:

Automatically adds __init__ and other methods.

Stores the file path where the trained model will be saved (artifacts/model.pkl).

🧠 Model Trainer Class

class ModelTrainer:

def __init__(self):

self.model_trainer_config=ModelTrainerConfig()

✔ Defines the ModelTrainer class:

Initializes with a configuration instance that holds the path to save the model.

🚀 Model Training Method

def initiate_model_trainer(self,train_array,test_array):

✔ Main method that trains and evaluates models.

Takes in train_array and test_array, which are NumPy arrays where:

All columns except the last one are features (X).

The last column is the target variable (y).

try:

logging.info("Split training and test input data")

✔ Logs a message saying it's about to split the data into X and y.

```
X_train,y_train,X_test,y_test=(
    train_array[:, :-1],
    train_array[:, -1],
    test_array[:, :-1],
    test_array[:, -1]
)
```

✔ Splits train_array and test_array:

X_train, X_test: All columns except the last (features).

y_train, y_test: The last column (target).

```
models = {
    "Random Forest": RandomForestRegressor(),
    "Decision Tree": DecisionTreeRegressor(),
    "Gradient Boosting": GradientBoostingRegressor(),
    "Linear Regression": LinearRegression(),
    "XGBRegressor": XGBRegressor(),
    "CatBoosting Regressor": CatBoostRegressor(verbose=False),
    "AdaBoost Regressor": AdaBoostRegressor(),
}
```

✔ Defines a dictionary of ML models to be trained and compared.

```
params={
    "Decision Tree": {
        'criterion':['squared_error', 'friedman_mse', 'absolute_error', 'poisson'],
        'splitter':['best','random'],
        'max_features':['sqrt','log2'],
    },
    "Random Forest":{
        'criterion':['squared_error', 'friedman_mse', 'absolute_error', 'poisson'],
        'n_estimators': [8,16,32,64,128,256]
    },
    "Gradient Boosting":{
        'loss':['squared_error', 'huber', 'absolute_error', 'quantile'],
        'learning_rate':[.1,.01,.05,.001],
        'subsample':[0.6,0.7,0.75,0.8,0.85,0.9],
        'criterion':['squared_error', 'friedman_mse'],
        'max_features':['auto','sqrt','log2'],
        'n_estimators': [8,16,32,64,128,256]
    },
}
```

```

"Linear Regression": {},
"XGBRegressor": {
    'learning_rate': [.1, .01, .05, .001],
    'n_estimators': [8, 16, 32, 64, 128, 256]
},
"CatBoosting Regressor": {
    'depth': [6, 8, 10],
    'learning_rate': [0.01, 0.05, 0.1],
    'iterations': [30, 50, 100]
},
"AdaBoost Regressor": {
    'learning_rate': [.1, .01, 0.5, .001],
    'loss': ['linear', 'square', 'exponential'],
    'n_estimators': [8, 16, 32, 64, 128, 256]
}
}

```

- ✓ Hyperparameter grid for each model (used for Grid Search or Random Search).

Some models like Linear Regression have no tuning parameters.

```

model_report=dict(evaluate_models(
    X_train=X_train,
    y_train=y_train,
    X_test=X_test,
    y_test=y_test,
    models=models,
    param=params
))

```

- ✓ Calls your custom evaluate_models() function:

Trains all models using their parameters.

Returns a dictionary of model names and their R² scores.

```

## To get best model score from dict
best_model_score = max(sorted(model_report.values()))

```

- ✓ Gets the highest R² score from the model_report.

```

## To get best model name from dict
best_model_name = list(model_report.keys())[
    list(model_report.values()).index(best_model_score)]

```

- ✓ Finds the name of the model that has the best score.

```
best_model = models[best_model_name]
```

- ✓ Retrieves the actual model object from the models dictionary.

```

if best_model_score < 0.6:
    raise CustomException("No best model found")

```

- ✓ If no model has an R² score above 0.6, it raises a custom exception.

```
logging.info(f"Best found model on both training and testing dataset")
```

- ✓ Logs that a good model has been found.

```

save_object(
    file_path=self.model_trainer_config.trained_model_file_path,
    obj=best_model
)

```

- ✓ Saves the best model to disk using the save_object utility function.

```
predicted=best_model.predict(X_test)
```

- ✓ Uses the best model to make predictions on test data.

```

r2_square = r2_score(y_test, predicted)
return r2_square

```

- ✓ Calculates and returns the R² score of the best model on the test set.

```

except Exception as e:
    raise CustomException(e, sys)

```

- ✓ Catches any errors in the training process and raises a CustomException for better debugging.

- ✓ Summary

This class does the following:

- Takes in training and testing arrays.
- Trains multiple models with hyperparameter tuning.
- Evaluates all models.
- Selects and saves the best model.
- Returns its R² score on test data.